

5 Databáze SIS-Junior (specializace DW)

Na jedné základní škole v ČR nasadili nový informační systém SIS-Junior pro evidenci žáků a jejich studijní agendy. V seznamu níže jsou uvedeny vybrané tabulky a sloupce, které se týkají dat potřebných pro tisk vysvědčení (uvažujeme tradiční vysvědčení, na kterém jsou předměty a známky, nikoli slovní hodnocení).

```
STUDENT(id, first_name:CHAR, last_name:CHAR, born:DATE)
```

```
CLASS(id, label:CHAR, academic_year:INT, school_year:INT{1..9})
```

```
SUBJECT(id, name:CHAR)
```

```
REPORT_GRADE(student_id, class_id, subject_id, semester:INT{1..2}, grade:INT{1..5})
```

Sloupce pojmenované id jsou PK (s hodnotou UUID), cizí klíče mají vždy tvar název-tabulky_id. Datové typy jsou naznačeny za názvy položek, u čísel (kde je to podstatné) jsou uvedeny i očekávané rozsahy. Akademický rok se ukládá jako číslo roku, kdy začal (tj. 2023 odpovídá roku 2023/24).

1. Uvedené DB schéma zcela jistě není úplné. Doplňte sloupce a tabulky (včetně popisu jejich sloupců), které nejsou ve výpisu výše, abychom měli úplný popis části schématu odpovídající záměru použití (agenda vysvědčení), především aby bylo možné zcela splnit následující body.

- Napište SQL dotaz typu **SELECT**, který vygeneruje podklady pro přípravu vysvědčení na druhé pololetí roku 2023/24 (tj. vypíše všechny platné trojice žák-třída-předmět, ke kterým pak učitelé budou ručně doplňovat známky a ukládat je do **REPORT_GRADE**).
- Napište SQL dotaz typu **SELECT**, který vypíše průměrné známky pro každou unikátní kombinaci předmětu, ročníku studentů a školního roku (známky obou pololetí se průměrují dohromady).

Nástin řešení 1) Na první pohled chybí spojení student-class (je to běžný M:N vztah, takže tabulka) a subject-class (taky M:N, co se učí v které třídě). Také je možné uvést v subject-class tabulce i jestli se daný předmět učí jen v prvním, druhém, nebo obou pololetích.

```
STUDENT_IN_CLASS(student_id, class_id) ...PK jsou oba FK
SUBJECT_IN_CLASS(subject_id, class_id, semester) ...pokud je v obou semestrech, znamená to 2 záznamy
```

- Stačí začít s **CLASS** a spojit přes pomocné tabulky (protože nás zajímají jen IDčka studenta a předmětu).

```
SELECT student_in_class.student_id, class.id, subject_in_class.subject_id FROM class
JOIN student_in_class ON class.id = student_in_class.class_id
JOIN subject_in_class ON class.id = subject_in_class.class_id
WHERE class.academic_year = '2023' AND subject_in_class.semester = 2
```

- Stačí jednoduchá kombinace **JOIN** a **GROUP BY**:

```
SELECT report_grade.subject_id, subject.name, class.academic_year, class.school_year,
       AVG(report_grade.grade)
FROM report_grade
JOIN class ON report_grade.class_id = class.id
JOIN subject ON report_grade.subject_id = subject.id
GROUP BY report_grade.subject_id, subject.name, class.academic_year, class.school_year
```

6 Databáze diplomových prací (specializace DW)

V rámci návrhu datového modelu aplikace byly na konceptuální úrovni identifikovány dvě datové třídy - *Osoba* a *Diplomová práce*. O osobě je nutné si pamatovat její unikátní číslo osoby, jméno, příjmení a typ (student/učitel). O diplomové práci je potřeba si pamatovat její název, studijní obor, název fakulty. Dále byly identifikovány následující vztahy:

- Diplomová práce má nejvýše jednoho *řešitele*. Řešitel může mít přiděleno nejvýše jednu práci
- Diplomová práce má právě jednoho *vedoucího*. Vedoucí může vést libovolný počet prací.

Pro výše popsanou situaci

- Načrtněte pro popsanou situaci odpovídající UML model. Nezapomeňte vyznačit kardinality vztahů.
- Převeďte Vámi navržený UML model na logický relační model. Nezapomeňte vyznačit všechny klíče a referenční integritu (cizí klíče). Převod proveďte tak, aby v cizích klíčích nemusela být nikdy vložena NULL hodnota.
- Je logický relační model vzniklý převodem UML modelu vhodný z hlediska dosažené normální formy, pokud víte, že v modelované doméně atribut název fakulty funkčně závisí na studijním oboru (tedy *Studijní obor* → *Název fakulty*)? Pokud ne, jak by měl být logický relační model upraven, aby vhodný byl a proč?

Nástin řešení

- | Osoba | | DiplomovaPrace |
|------------|------------|----------------|
| - Cislo | (0,1) Resi | (0,1) |
| - Jmeno | | - Nazev |
| - Prijmeni | 1 Vede | (0,*) |
| - Typ | | - StudijniObor |
| | | - NazevFakulty |
- Osoba(OsobaID, Cislo, Jmeno, Prijmeni, Typ)

- $\text{DiplomovaPrace}(\underline{\text{DiplomovaPraceID}}, \text{Nazev}, \text{StudijniObor}, \text{NazevFakulty}, \text{OsobaID})$
 $\text{OsobaID} \subseteq \text{Osoba.OsobaID}$
- $\text{Resi}(\underline{\text{OsobaID}}, \underline{\text{DiplomovaPraceID}})$,
 $\text{OsobaID} \subseteq \text{Osoba.OsobaID}$,
 $\text{DiplomovaPraceID} \subseteq \text{DiplomovaPrace.DiplomovaPraceID}$

Kardinalita vztahů se projeví v rozdílné definici klíčů vztahových tabulek.

- Vzhledem k tranzitivní závislosti $\text{DiplomovaPraceID} \rightarrow \text{StudijniObor} \rightarrow \text{NazevFakulty}$ není tabulka DiplomovaPrace ve 3NF. Pro nápravu je potřeba tabulku dekomponovat podle závislosti $\text{StudijniObor} \rightarrow \text{NazevFakulty}$. Vzniknou dvě menší tabulky
 - $\text{Obor}(\underline{\text{StudijniObor}}, \text{NazevFakulty})$
 - $\text{DiplomovaPrace}(\underline{\text{DiplomovaPraceID}}, \text{Nazev}, \text{StudijniObor}), \text{StudijniObor} \subseteq \text{Obor.StudijniObor}$

7 Komprese (specializace DW)

Zpráva nad abecedou $\{a, b, c, d, e\}$ je náhodnou veličinou s rozložením pravděpodobností $\{\frac{4}{20}, \frac{3}{20}, \frac{2}{20}, \frac{4}{20}, \frac{7}{20}\}$.

- Vytvořte odpovídající kódovací strom pro Huffmanovo kódování do binární abecedy $\{0, 1\}$. Sourozence ve stromu řaďte dle (kumulativní) pravděpodobnosti. Následně zakódujte začátek zprávy *cdebea...*
- Nadřízení by si přáli, aby byly zprávy kódovány pomocí následující tabulky:
 $a = 1, b = 10, c = 100, d = 1000, e = 0000$.
 Vysvětlete, proč použití tohoto způsobu kódování není úplně vhodné.
- Jaké binární kódování by bylo potřeba použít, aby se délka zpráv co nejvíce blížila entropii zprávy?

Nástin řešení

- Strom se konstruuje zdola nahoru spojováním dvojic symbolů s nejnižší pravděpodobností výskytu. Nejprve se tedy spojí symboly (c, b) s celkovou pravděpodobností $\frac{5}{20}$. Poté symboly (a, d) s celkovou pravděpodobností $\frac{8}{20}$. Poté symboly $((c, b), e)$ s celkovou pravděpodobností $\frac{12}{20}$. Nakonec symboly $((a, d), ((c, b), e))$ s celkovou pravděpodobností $\frac{20}{20}$. Kódovací strom tedy přidělí jednotlivým znakům kódy:
 $a = 00, d = 01, c = 100, b = 101, e = 11$.
 Začátek zprávy se v takovém případě zakóduje jako $100|01|11|101|11|00|...$
- Snazší argument je, že kódy svými délkami neodpovídají rozložení pravděpodobnosti (mj. dva poslední znaky se dvěma nejčastějšími výskyty mají nejdelší kódy) a kódové zprávy budou proto delší než v případě Huffmanova kódu. Bonusový postřeh je, že kód není prefixový, ale postfixový, a může proto nastat problém při jeho dekódování zleva doprava. Konkrétně například úseky $100000...00|1...$ bude potřeba nejprve celé přečíst a dekódovat zprava doleva, aby se zjistilo, jakým znakem zpráva začíná.
- Aritmetické kódování.

8 Odeslání webového formuláře (specializace DW)

Mějme HTML formulář v tradiční (CGI-like, tedy bez skriptů na straně klienta) webové aplikaci na přidávání položek do databáze.

```
<form id="addForm" action="index.php?action=addItem" method="POST">...
```

Skript na straně serveru zpracuje data z formuláře a pokud jsou korektní, přidá nový záznam do databáze. Jako odpověď pak vygeneruje HTML stránku, na které je tabulka se všemi záznamy v databázi.

1. Uživatel vyplnil formulář, odeslal jej na server a zobrazila se mu stránka s aktuální tabulkou všech záznamů (včetně nově přidaného). Co přesně se stane, když uživatel klikne na tlačítko pro znovunačtení stránky (*Refresh*) v prohlížeči? Takové chování zřejmě není zcela žádoucí z pohledu uživatele, navrhněte příslušnou úpravu aplikace (bez použití skriptů na straně klienta).
2. Pokud data odesílaného formuláře nejsou korektní (nelze je uložit), je třeba zobrazit tento formulář znovu a zároveň jej předvyplnit daty, která uživatel zadal (aby je uživatel mohl jen upravit a nemusel zadávat znovu). Jak tuto funkcionalitu zajistit v kombinaci s vaším řešením problému z bodu 1?
3. Jak by bylo možné situaci popsanou v prvním bodě vyřešit, pokud bychom dovolili použít skripty na straně klienta? Načrtněte kostru takového řešení v JavaScriptu, pokud si nejste jisti názvy funkcí nebo událostí z DOM API, doplňte k nim komentář s vysvětlením.

Vaše řešení z bodů 1. a 2. popište pokud možno stručně a strukturovaně (např. formou odrážek), stačí popis pro běžnou situaci (není třeba pokrývat všechny myslitelné speciální případy). Naopak buďte konkrétní v technických detailech, zejména těch, které se týkají ukládání a přenosu dat mezi klientem a serverem (pokud je např. potřeba data od uživatele v bodu 2. dočasně někam uložit, napište kam/jak). Také můžete použít fragmenty PHP kódu, kde je považujete za vhodné.

Nástin řešení 1. Prohlížeč se zeptá, jestli má znovu odeslat data z formuláře, a pokud uživatel potvrdí, znovu se položka přidá do DB (tj. bude tam 2x). Vhodné řešení je místo HTML stránky vrátit HTTP response 302 nebo 303 (redirect) a hlavičku `Location` s novým URL (třeba i sám na sebe). Request na nové URL bude vykonán metodou GET a nové URL jen vrátí HTML (pak s refreshem nebude problém).

2. Rozdělení dotazu v bodě 1. na dva dotazy (POST uloží a udělá redirect, GET načte stránku) vede k tomu, že odeslaná data (POST) je potřeba někam uložit, aby si je následný GET mohl načíst. Typicky se na to používá user session (PHP na to má přímo API), jejíž ID je uloženo v cookies. Alternativně lze použít databázi nebo soubory. V obou případech je třeba, aby měla “ta data” nějaký unikátní identifikátor (např. náhodný nebo UUID), který se zároveň přenese v URL (GET na ten formulář bude obsahovat navíc query parametr s tím identifikátorem, tím se zároveň řekne obsluze toho GETu, že nemá zobrazit prázdný formulář, ale předvyplněný + zvýraznit chyby). HTML/JS testování formuláře před odesláním problém neřeší, testovat se musí na serveru.

3. Pokud dovolíme JS, můžeme si celou situaci usnadnit tak, že za odeslání formuláře je zodpovědný asynchronní HTTP request. Kostra takového JS by vypadala zhruba takto:

```
const form = document.getElementById('addForm');
form.onSubmit = function(ev) {
  ev.preventDefault();
  fetch(form.action, {
    body: new FormData(form),
    method: "POST",
  }).then(res => res.json())
  .then(res => {
    if (isResponseOk(res)) {
      location.assign('URL to page where list is displayed');
    } else {
      showErrors(form, res);
    }
  })
};
```

Funkce `isResponseOk()` a `showErrors()` není z pohledu otázky třeba rozepisovat, jejich semantika je jasná z názvu.